



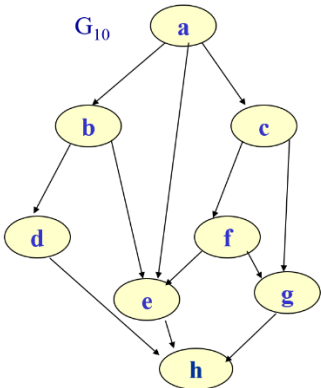
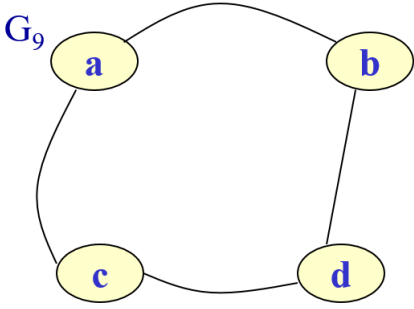
Implementação

Busca em Largura e Profundidade (Grupo do Projeto)

Nome do Integrante (ordem alfabética)	RA
André Doerner Duarte	10427938
Matheus Leonardo Cardoso Kroeff	10426434
Naoto Ushizaki	10437445

Tendo por base a implementação da classe TGrafo para matriz de adjacência ou lista de adjacência e o estudo realizado sobre busca em profundidade e largura em aula, realizar a implementação dos dois algoritmos estudados na classe TGrafo. Utilizar como subsídios os métodos constantes no material da aula.

Os resultados de pelo menos dois dos seus testes devem ser colocados no relatório a seguir. Para os testes, fazer uso dos dois exemplos do material da aula. Nos testes, devem ser utilizados os exemplos de Grafo direcionado e não direcionado que constam no material da aula, conforme destacados a seguir.

Grafos a serem utilizados para testes (Necessariamente)	
	
Grafo Direcionado	Grafo não Direcionado

Além disso, ao final do relatório (Apêndice), colocar o código fonte dos métodos implementados para esta atividade.

Encaminhar o relatório e todos os arquivos com código fonte compactados ao enviar pela área do Ambiente Virtual na data limite programada.



Resultado do teste realizado:

```
dededuarte@pop-os:~/Facul/6s/BDFS$ make
dededuarte@pop-os:~/Facul/6s/BDFS$ ./a.out
===== Grafos Gerados =====
Gafo G1:
    0 1 2 3 4 5 6 7
0 [0 1 1 0 1 0 0 0]
1 [0 0 0 1 1 0 0 0]
2 [0 0 0 0 0 1 1 0]
3 [0 0 0 0 0 0 0 1]
4 [0 0 0 0 0 0 0 1]
5 [0 0 0 0 1 0 1 0]
6 [0 0 0 0 0 0 0 1]
7 [0 0 0 0 0 0 0 0]

Gafo G2:
    0 1 2 3
0 [0 1 1 0]
1 [1 0 0 1]
2 [1 0 0 1]
3 [0 1 1 0]

===== Teste Busca em Profundidade =====
Ordem dos nos visitados de G1:
[0, 1, 3, 7, 4, 2, 5, 6]

Ordem dos nos visitados de G2:
[0, 1, 3, 2]

===== Teste Busca em Largura =====
Ordem dos nos visitados de G1:
[0, 1, 2, 4, 3, 5, 6, 7]

Ordem dos nos visitados de G2:
[0, 1, 2, 3]
espaco liberado
espaco liberado
```



GitHub da atividade: https://github.com/DedeDuarte/Grafo-Implementacao_Busca_Largura_Profundidade.git

Algoritmo de busca em profundidade:

```
std::vector<int> TGrafo::busca_profundidade(int inicial) {
    std::vector<int> visitados;
    std::stack<int> stack;
    int no = inicial;

    visitados.push_back(no);
    stack.push(no);

    while (!stack.empty()) {
        no = stack.top();
        stack.pop();

        for (int i = 0; i < this->n; i++) {
            if (this->adj[no][i] == 0 || _vector_find(visitados, i))
                continue;

            visitados.push_back(i);
            stack.push(no);

            no = i;
        }
    }

    return visitados;
}
```



Algoritmo de busca em largura:

```
std::vector<int> TGrafo::busca_largura(int inicial) {
    std::vector<int> visitados;
    std::queue<int> queue;
    int no = inicial;

    visitados.push_back(no);
    queue.push(no);

    while (!queue.empty()) {
        no = queue.front();
        queue.pop();

        for (int i = 0; i < this->n; i++) {
            if (this->adj[no][i] == 0 || _vector_find(visitados, i))
                continue;

            visitados.push_back(i);
            queue.push(i);
        }
    }

    return visitados;
}
```



Outras funções criadas:

```
void preencher_grafos(TGrafo& g1, TGrafo& g2) {
    // Criando vertices para G1
    g1.insereA(0, 1);
    g1.insereA(0, 2);
    g1.insereA(0, 4);
    g1.insereA(1, 3);
    g1.insereA(1, 4);
    g1.insereA(2, 5);
    g1.insereA(2, 6);
    g1.insereA(3, 7);
    g1.insereA(4, 7);
    g1.insereA(5, 4);
    g1.insereA(5, 6); // Six Seven hehe
    g1.insereA(6, 7);

    // Criando vertices para G2
    g2.insereA(0, 1);
    g2.insereA(0, 2);
    g2.insereA(1, 0);
    g2.insereA(1, 3);
    g2.insereA(2, 0);
    g2.insereA(2, 3);
    g2.insereA(3, 1);
    g2.insereA(3, 2);
}

void print_vector_int(std::vector<int> v) {
    std::cout << "[";

    int i;
    for (i = 0; i < (int)v.size()-1; i++)
        std::cout << v[i] << ", ";

    std::cout << v[i] << "]" << std::endl;
}
```



Main:

```
int main() {
    // Criando grafos
    TGrafo g1(8);
    TGrafo g2(4);

    preencher_grafos(g1, g2);

    // Conferindo grafos
    std::cout << "==== Grafos Gerados =====>< std::endl;
    std::cout << "Grafo G1:" << std::endl;
    g1.show();

    std::cout << "\nGrafo G2:" << std::endl;
    g2.show();

    // Busca em profundidade
    std::vector<int> visitados_g1_p = g1.busca_profundidade(0);
    std::vector<int> visitados_g2_p = g2.busca_profundidade(0);

    std::cout << "\n==== Teste Busca em Profundidade =====>< std::endl;
    std::cout << "Ordem dos nos visitados de G1:" << std::endl;
    print_vector_int(visitados_g1_p);

    std::cout << "\nOrdem dos nos visitados de G2:" << std::endl;
    print_vector_int(visitados_g2_p);

    // Busca em largura
    std::vector<int> visitados_g1_l = g1.busca_largura(0);
    std::vector<int> visitados_g2_l = g2.busca_largura(0);

    std::cout << "\n==== Teste Busca em Largura =====>< std::endl;
    std::cout << "Ordem dos nos visitados de G1:" << std::endl;
    print_vector_int(visitados_g1_l);

    std::cout << "\nOrdem dos nos visitados de G2:" << std::endl;
    print_vector_int(visitados_g2_l);

    return 0;
}
```